

EXAMEN TEÓRICO - BANCO DE 50 PREGUNTAS

*Spring Boot · REST · Spring MVC · JPA/Hibernate · Arquitectura Hexagonal · Relaciones 1:N
· Errores · Transacciones*

Capítulos 04 al 08 - ParkFlow 360

Estudiante	_____
Fecha	_____
Puntuación	_____ / 100
Tiempo	Definido por el docente

Objetivo de la evaluación

Verificar que el estudiante domina el mapa conceptual integrador y posee la base teórica necesaria para desarrollar, explicar y defender los Capítulos 04, 05, 06, 07 y 08 de ParkFlow 360 sin confundir responsabilidades, tecnologías, patrones ni capas arquitectónicas.

Instrucciones para el estudiante

- Responda con explicación conceptual completa. No se aceptan respuestas de una sola palabra o simples definiciones memorizadas.
- Cuando la pregunta solicite comparar, indique diferencias, responsabilidades y relación entre los conceptos.
- Cuando se solicite un flujo, explique el orden de intervención de los componentes y la dirección de las dependencias.
- No es necesario escribir código, salvo que el docente lo solicite posteriormente como ampliación.
- Cada pregunta puede evaluarse sobre 2 puntos para obtener una escala total de 100 puntos.

Dinámica de evaluación con ruleta online

El profesor utilizará una ruleta online que contendrá las 50 preguntas de este banco. La pregunta seleccionada al azar será la única que el estudiante deberá responder.

1. Selección: el docente ejecutará la ruleta y la pregunta obtenida será la pregunta oficial para ese estudiante.

2. Una sola pregunta: cada estudiante responderá únicamente una de las 50 preguntas.

3. Condición: la respuesta debe ser correcta, clara, conceptualmente completa y demostrar comprensión; no se aceptará una respuesta basada solo en memorización.

4. Beneficio: si responde correctamente, el estudiante obtiene 5 puntos extra.

5. Uso de los puntos: los 5 puntos extra podrán utilizarse en cualquier evaluación oficial del semestre, de acuerdo con el registro y control del docente.

6. Respuesta insuficiente: si la respuesta es incorrecta o incompleta, no se otorgan los 5 puntos extra y no se modifica ninguna evaluación oficial.

Criterio docente: la respuesta se considerará satisfactoria cuando el estudiante pueda definir el concepto, explicar su función, relacionarlo con otros elementos del mapa conceptual y vincularlo con los Capítulos 04 al 08 cuando corresponda.

Importante: como la ruleta puede seleccionar cualquiera de las 50 preguntas, el estudiante debe estudiar y completar todo el mapa conceptual, no solamente un capítulo o una sección.

Cobertura del examen

Bloque	Preguntas	Cobertura principal
Capítulo 04	1-10	HTTP, REST, API, Spring MVC, DTO, JSON, Validation, Postman
Capítulo 05	11-20	PostgreSQL, JDBC, ORM, JPA, Hibernate, Spring Data JPA, DataGrip
Capítulo 06	21-30	Hexagonal, IoC/DI, Ports, Use Cases, Adapters, mappers, dependencias
Capítulo 07	31-40	Cliente 1:N Vehículo, PK/FK, @ManyToOne, LAZY, colaboración modular
Capítulo 08	41-50	Errores, validaciones, Advice, códigos HTTP, transacciones

CAPÍTULO 04 - Fundamentos Web, REST, Spring MVC, APIs, DTOs, Validation y herramientas cliente

Preguntas teóricas. Responda demostrando comprensión, relación entre conceptos y criterio arquitectónico.

1. Explique qué es una API y diferénciela de un endpoint. Utilice un ejemplo conceptual de ParkFlow 360 para demostrar la relación entre ambos términos.
2. Explique qué es HTTP y describa los componentes esenciales de una petición HTTP: método, URL, headers, parámetros y body. ¿Qué función cumple cada uno?
3. Defina REST como estilo arquitectónico. Explique por qué REST no significa simplemente “usar JSON” ni tampoco es sinónimo de CRUD.
4. Explique el principio stateless de REST. ¿Qué implica para el servidor y para cada petición enviada por un cliente?
5. Compare los métodos HTTP GET, POST, PUT y DELETE desde el punto de vista de intención semántica e idempotencia. ¿Por qué POST normalmente no se considera idempotente?
6. Explique la diferencia entre los códigos HTTP 200 OK, 201 Created, 204 No Content, 400 Bad Request y 404 Not Found. Indique cuándo corresponde utilizar cada uno en una API REST.
7. Explique qué función cumple Spring MVC dentro de una aplicación Spring Boot y describa conceptualmente el recorrido de una petición desde que llega al backend hasta que un Controller la atiende.
8. Diferencie @RestController, @RequestMapping, @GetMapping y @PostMapping. Explique qué responsabilidad expresa cada anotación dentro de un Controller REST.
9. Compare @PathVariable, @RequestParam y @RequestBody. ¿Qué parte de la petición HTTP representa cada uno y en qué situaciones se debería utilizar?
10. Explique por qué una API profesional separa Request DTO, modelo de dominio y Response DTO. Incluya en su respuesta el papel de Jackson, @Valid, Jakarta Validation, Postman e IntelliJ HTTP Client dentro de este flujo.

CAPÍTULO 05 - PostgreSQL, JDBC, ORM, JPA, Hibernate y Spring Data JPA

Preguntas teóricas. Responda demostrando comprensión, relación entre conceptos y criterio arquitectónico.

11. Explique la cadena tecnológica JDBC -> JPA -> Hibernate -> Spring Data JPA -> PostgreSQL. ¿Qué responsabilidad cumple cada elemento y por qué no deben tratarse como sinónimos?
12. Defina ORM y explique qué problema intenta resolver al trabajar con objetos Java y tablas relacionales. ¿Por qué utilizar ORM no elimina la necesidad de comprender SQL y diseño de bases de datos?
13. Explique qué es JPA y qué es Hibernate. ¿Cuál es la relación entre una especificación y una implementación en este contexto?
14. Explique qué aporta Spring Data JPA sobre JPA/Hibernate y por qué un JpaRepository no debe confundirse con el modelo de dominio.
15. Explique el significado arquitectónico y operativo de configurar hibernate.ddl-auto=validate cuando la base de datos ya fue diseñada previamente. ¿Por qué create o create-drop serían decisiones riesgosas en este escenario?
16. Explique qué es un schema en PostgreSQL y por qué no basta con conectarse a la base de datos correcta si Hibernate apunta a un schema equivocado.
17. Explique la función de @Entity, @Table, @Id, @GeneratedValue y @Column en una entidad de persistencia. ¿Qué información de la tabla física representa cada anotación?
18. Explique por qué ClienteJpaEntity y Cliente de dominio deben ser clases distintas en un diseño desacoplado. ¿Qué tipo de cambios podría absorber cada una sin afectar innecesariamente a la otra?
19. Compare la declaración unique=true en @Column con una restricción UNIQUE real en PostgreSQL. ¿Cuál constituye la última barrera de integridad y por qué?
20. Explique cómo comprobar que un POST realmente persistió información. Relacione en una sola respuesta la evidencia obtenida desde la respuesta HTTP, los logs SQL de Hibernate, DataGrip y la fila física en PostgreSQL.

CAPÍTULO 06 - Arquitectura hexagonal, IoC/DI, puertos, casos de uso, adaptadores y patrones

Preguntas teóricas. Responda demostrando comprensión, relación entre conceptos y criterio arquitectónico.

21. Explique el objetivo central de la arquitectura hexagonal y la regla de dirección de dependencias. ¿Qué significa que las dependencias deben apuntar hacia el núcleo?
22. Diferencie dominio, capa de aplicación e infraestructura. ¿Qué tipo de responsabilidades deberían vivir en cada una?
23. Explique qué es un Port IN y qué representa en términos de capacidades del sistema. ¿Por qué un Port IN no debería mencionar HTTP, JSON, Controller ni Spring Data JPA?
24. Explique qué es un Port OUT y por qué ClienteRepositoryPort no debería extender JpaRepository. ¿Qué ventaja se obtiene al definir el contrato desde el núcleo?
25. Diferencie un Controller de un Port IN. Aunque ambos participan en la entrada al sistema, ¿qué responsabilidad específica tiene cada uno?
26. Explique qué función cumple un Application Service o caso de uso. ¿Por qué debe depender de abstracciones y no directamente de SpringDataClienteRepository?
27. Explique qué es un Adapter IN y qué es un Adapter OUT. Use ClienteController y ClientePersistenceAdapter como ejemplos conceptuales.
28. Explique el principio de Inversión de Dependencias aplicado a ParkFlow 360. ¿Cómo se relacionan IoC, Dependency Injection y la inyección por constructor con la arquitectura hexagonal?
29. Explique el papel de los mappers en una arquitectura hexagonal. ¿Por qué son necesarias traducciones separadas entre Web DTO <-> Domain y Domain <-> JPA Entity?
30. Analice la siguiente afirmación: “Si ClienteController inyecta directamente SpringDataClienteRepository, la aplicación puede funcionar, pero el diseño arquitectónico es incorrecto”. Explique con detalle por qué.

CAPÍTULO 07 - Relación Cliente 1:N Vehículo entre módulos

Preguntas teóricas. Responda demostrando comprensión, relación entre conceptos y criterio arquitectónico.

31. Explique qué significa una relación 1:N entre Cliente y Vehículo. Identifique cuál es la entidad padre, cuál es la dependiente y dónde se ubica físicamente la clave foránea.
32. Diferencie PK y FK. ¿Qué problema de integridad resuelve una FOREIGN KEY en PostgreSQL dentro de la relación Cliente-Vehículo?
33. Explique por qué la misma relación Cliente 1:N Vehículo se representa de manera diferente en dominio, persistencia JPA y PostgreSQL.
34. Explique por qué el dominio Vehículo puede almacenar clienteId sin almacenar ClienteJpaEntity. ¿Qué principio arquitectónico protege esta decisión?
35. Explique qué representan @ManyToOne y @JoinColumn(name="cliente_id") en VehiculoJpaEntity. Aclare por qué estas anotaciones no son la FK física.
36. Explique FetchType.LAZY y justifique por qué puede ser una decisión adecuada para la relación muchos-a-uno de Vehículo hacia Cliente.
37. Explique por qué VehiculoService debe validar que el cliente exista antes de intentar persistir el vehículo, aun cuando PostgreSQL ya tenga una FOREIGN KEY.
38. Explique por qué el módulo Vehicle debería consultar a Customer mediante ConsultarClienteUseCase y no mediante SpringDataClienteRepository. ¿Qué tipo de acoplamiento se evita?
39. Explique la diferencia entre una validación de negocio como “la placa no debe estar duplicada” y una restricción UNIQUE en PostgreSQL. ¿Por qué es razonable mantener ambas?
40. Explique conceptualmente el recorrido completo para registrar un vehículo: desde el cliente HTTP hasta la FK cliente_id en PostgreSQL y el retorno de la respuesta. Señale en qué capa intervienen DTO, Port IN, caso de uso, Port OUT, adapter, JPA y PostgreSQL.

CAPÍTULO 08 - Errores profesionales, validaciones de negocio y transacciones

Preguntas teóricas. Responda demostrando comprensión, relación entre conceptos y criterio arquitectónico.

41. Explique por qué una API profesional necesita un contrato de error consistente. ¿Qué ventajas aporta un objeto como `ApiError` para frontend, pruebas y diagnóstico?
42. Diferencie validación de formato, validación de negocio e integridad física de base de datos. Proporcione un ejemplo conceptual de cada una en ParkFlow 360.
43. Explique qué función cumple `@RestControllerAdvice` y por qué evita repetir bloques `try/catch` en cada Controller.
44. Explique qué función cumple `@ExceptionHandler` y cómo permite traducir una excepción interna del sistema a una respuesta HTTP con semántica adecuada.
45. Compare los códigos 400 Bad Request, 404 Not Found, 409 Conflict y 500 Internal Server Error. ¿Qué tipo de situación debería representar cada uno?
46. Explique por qué una excepción de negocio, como `PlacaDuplicadaException`, no debería contener `ResponseEntity` ni conocer conceptos HTTP.
47. Explique por qué capturar `Exception` de forma genérica en todos los Controllers constituye una mala práctica de diseño y mantenimiento.
48. Defina una transacción y explique las propiedades conceptuales que se buscan cuando varias operaciones deben confirmarse juntas o revertirse juntas.
49. Explique qué función cumple `@Transactional` y por qué el límite transaccional natural suele ubicarse en un caso de uso de escritura o `Application Service`, no en el Controller.
50. Explique el flujo completo de error en ParkFlow 360: desde una petición inválida o un conflicto de negocio hasta `GlobalExceptionHandler`, `ApiError` y el código HTTP final. Relacione también qué ocurre si el fallo proviene de una constraint de PostgreSQL.

Checklist de cobertura para el docente

- HTTP, REST, API, endpoint, JSON, métodos y códigos HTTP.
- Spring Framework, Spring Boot, Spring MVC, Controller, DispatcherServlet conceptual e IoC/DI.
- DTOs, Jackson, Jakarta Validation, @Valid y herramientas cliente de prueba.
- PostgreSQL, schemas, JDBC, ORM, JPA, Hibernate, Spring Data JPA y evidencia con DataGrip.
- Arquitectura hexagonal: dominio, aplicación, infraestructura, Ports IN/OUT, Use Cases y Adapters.
- Principios/patrones: inversión de dependencias, Dependency Injection, Repository, Adapter y Mapper.
- Relación 1:N Cliente-Vehículo en dominio, JPA y PostgreSQL; PK, FK, @ManyToOne, @JoinColumn y LAZY.
- Validación de entrada, negocio e integridad de base de datos.
- ApiError, @RestControllerAdvice, @ExceptionHandler y semántica 400/404/409/500.
- @Transactional y frontera transaccional del caso de uso.
- Flujos end-to-end completos tanto de éxito como de error.

Documento docente - ParkFlow 360 · Capítulos 04-08